INSTITUTO TECNOLÓGICO SUPERIOR DE MONCLOVA

Tarea 2.2

Transición de diagrama E-R a notación UML

Comparativa E-R vs. UML y análisis para el desarrollo orientado a objetos

Unidad 2: Diseño de bases de datos con el modelo E-R

Materia: Fundamentos de Bases de Datos

Alumno: Edgar Alejandro Hernández Guajardo

Matrícula: I24050409

Docente: Ing. Ruben Riojas

1. Introducción

Una vez elaborado el diagrama conceptual E-R de un sistema, es común que su implementación se haga con lenguajes orientados a objetos que se conectan a una base de datos. Para pasar del diseño de los datos al diseño del software, el diagrama E-R puede transformarse en un diagrama de clases de UML (Lenguaje Unificado de Modelado), que conserva la estructura de la información y añade el comportamiento de los objetos.

Este documento toma el diagrama E-R extendido de la Tarea 2.1 (sistema de gestión de una biblioteca) y lo convierte en un diagrama de clases UML. Establece la correspondencia entre entidades y clases, atributos y propiedades, y relaciones y asociaciones (composición, agregación y herencia); presenta una comparativa detallada entre ambos modelos y analiza sus coincidencias y diferencias para el desarrollo orientado a objetos. El diagrama UML fue elaborado con Mermaid, cuyo código fuente es editable.

2. Marco conceptual

2.1 Modelo Entidad-Relación

El modelo E-R, propuesto por Chen (1976), describe los datos mediante entidades, atributos y relaciones. Su extensión (E-RE) incorpora la especialización, la generalización y la agregación (Elmasri y Navathe, 2016). Es un modelo conceptual orientado al diseño de bases de datos y no incluye el comportamiento de los objetos.

2.2 Diagrama de clases de UML

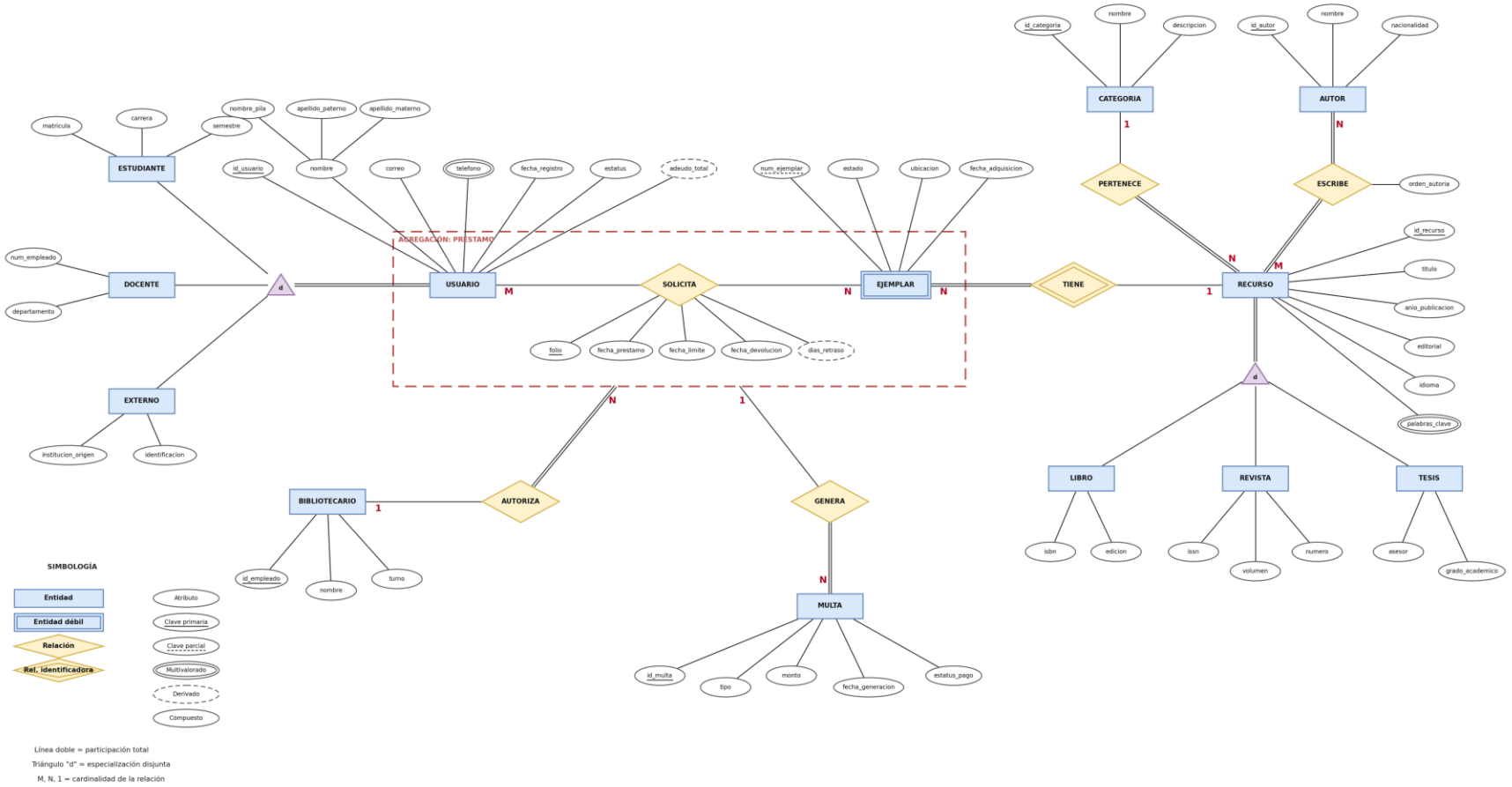
UML es un lenguaje estándar de modelado de sistemas de software mantenido por el Object Management Group (2017). Su diagrama de clases muestra las clases del sistema, con sus propiedades y operaciones, y las relaciones entre ellas: asociación, agregación, composición y generalización (Booch et al., 2005; Fowler, 2004). A diferencia del E-R, describe tanto la estructura como el comportamiento.

3. Diagrama E-R de origen

La Figura 1 muestra el diagrama E-R extendido elaborado en la Tarea 2.1, que es el punto de partida de la transformación. Incluye una entidad débil (EJEMPLAR), dos jerarquías de especialización (USUARIO y RECURSO) y una agregación (PRÉSTAMO).

Figura 1

Diagrama E-R extendido del sistema de biblioteca (Tarea 2.1)



Nota. Elaboración propia con draw.io (Tarea 2.1).

4. Reglas de transformación de E-R a UML

4.1 Correspondencia general de conceptos

La Tabla 1 resume las reglas aplicadas para convertir cada concepto del modelo E-R/E-RE en su equivalente de UML (Teorey et al., 2011; Connolly y Begg, 2015).

Tabla 1

Correspondencia entre conceptos del modelo E-R y de UML

Concepto del modelo E-R / E-RE	Elemento en UML (diagrama de clases)	Observación
Entidad	Clase	Los atributos de la entidad se convierten en propiedades de la clase y se agregan operaciones (comportamiento).
Entidad débil y relación identificadora	Clase unida por composición (rombo relleno)	La parte no existe sin el todo: el ciclo de vida de la clase débil depende de la clase propietaria.
Atributo simple	Propiedad con tipo y visibilidad	UML exige declarar el tipo de dato y la visibilidad (-, +, #).
Atributo clave	Propiedad identificadora	En orientación a objetos cada objeto tiene identidad propia; la clave se conserva como propiedad para el mapeo a la base de datos.
Atributo compuesto	Clase «dataType» relacionada por composición	Sus componentes pasan a ser propiedades de esa clase.
Atributo multivalorado	Propiedad con multiplicidad [1..*] o [0..*]	Se implementa como colección o arreglo.
Atributo derivado	Propiedad derivada (/) y operación de cálculo	Se calcula en lugar de almacenarse.
Relación binaria 1:N	Asociación con multiplicidades	Cada extremo lleva su multiplicidad.
Relación M:N o con atributos	Clase de asociación o clase intermedia	Los atributos de la relación pasan a esa clase.
Cardinalidad y participación	Multiplicidad mínimo..máximo	Participación total equivale a mínimo 1; parcial equivale a mínimo 0.
Especialización / generalización	Generalización (herencia)	Se anotan las restricciones {disjoint} y {complete}.
Agregación del E-RE (relación tratada como entidad)	Clase con sus asociaciones	La relación agregada se vuelve una clase que otras clases pueden asociar.
Dominio de valores (lista cerrada)	Enumeración «enumeration»	No existe símbolo equivalente en el E-R básico.

Nota. Elaboración propia a partir de Chen (1976), Booch et al. (2005) y Elmasri y Navathe (2016).

4.2 Entidades y clases

Tabla 2

Correspondencia entre entidades del E-R y clases de UML

Entidad / elemento E-R	Clase UML	Tipo y observación
USUARIO	Usuario	Clase abstracta (supertipo de la jerarquía)

Entidad / elemento E-R	Clase UML	Tipo y observación
ESTUDIANTE, DOCENTE, EXTERNO	Estudiante, Docente, Externo	Clases hijas de Usuario (herencia)
RECURSO	Recurso	Clase abstracta (supertipo de la jerarquía)
LIBRO, REVISTA, TESIS	Libro, Revista, Tesis	Clases hijas de Recurso (herencia)
EJEMPLAR (entidad débil)	Ejemplar	Parte de Recurso (composición)
AUTOR	Autor	Clase independiente
CATEGORIA	Categoria	Clase que agrupa recursos (agregación)
BIBLIOTECARIO	Bibliotecario	Clase independiente
MULTA	Multa	Parte de Prestamo (composición)
SOLICITA agregada como PRÉSTAMO	Prestamo	La agregación se convierte en clase con folio y fechas
Atributo compuesto nombre	Nombre	Clase «dataType» compuesta por Usuario
Relación ESCRIBE (M:N con atributo)	Autoria	Clase de asociación entre Recurso y Autor
Valores de estado y de tipo de multa	EstadoEjemplar, TipoMulta	Enumeraciones (elementos nuevos de UML)

Nota. Elaboración propia.

4.3 Atributos y propiedades

Tabla 3

Correspondencia entre atributos del E-R y propiedades de UML

Tipo de atributo	Atributo en el E-R	Propiedad en UML
Clave	id_usuario, id_recurso, id_autor, id_categoria, id_empleado, id_multa, folio	-idUsuario : String, -idRecurso : String, etc.
Clave parcial	num_ejemplar	-numEjemplar : Integer (se identifica dentro de su Recurso)
Compuesto	nombre (nombre_pila, apellido_paterno, apellido_materno)	-nombre : Nombre; la clase Nombre contiene las tres propiedades
Multivalorado	telefono, palabras_clave	-telefonos : String[1..*], -palabrasClave : String[0..*]
Derivado	adeudo_total, dias_retraso	-/adeudoTotal : Decimal y -/diasRetraso : Integer, con las operaciones calcularAdeudo() y calcularDiasRetraso()
De relación	orden_autoria; fechas del préstamo	-ordenAutoria : Integer en Autoria; -fechaPrestamo, -fechaLimite y -fechaDevolucion en Prestamo
De dominio cerrado	estado, tipo	-estado : EstadoEjemplar y -tipo : TipoMulta (enumeraciones)

Nota. Elaboración propia. El prefijo "/" indica una propiedad derivada.

4.4 Relaciones y asociaciones

Las cardinalidades y las restricciones de participación del E-R se expresan en UML mediante multiplicidades: una participación total se traduce en un mínimo de 1 y una participación parcial

en un mínimo de 0. Las relaciones con características de todo-parte se modelaron como composición o agregación, y las especializaciones como herencia.

Tabla 4

Correspondencia entre relaciones del E-R y asociaciones de UML

Relación E-R	Cardinalidad y participación (E-R)	Elemento UML	Multiplicidades
ES (USUARIO)	Especialización total y disjunta	Generalización (herencia)	Usuario ◁— Estudiante, Docente, Externo {disjoint, complete}
ES (RECURSO)	Especialización total y disjunta	Generalización (herencia)	Recurso ◁— Libro, Revista, Tesis {disjoint, complete}
TIENE (identificadora)	1:N; EJEMPLAR total, RECURSO parcial	Composición	Recurso 1 ◆— 0..* Ejemplar
PERTENECE	N:1; RECURSO total, CATEGORIA parcial	Agregación	Categoria 1 ◇— 0..* Recurso
ESCRIBE	M:N; ambos totales; atributo orden_autoria	Clase de asociación	Recurso 1 — 1..* Autoria; Autor 1 — 1..* Autoria
SOLICITA (PRÉSTAMO)	M:N; ambos parciales	Asociaciones hacia la clase Prestamo	Usuario 1 — 0..* Prestamo; Ejemplar 1 — 0..* Prestamo
AUTORIZA	1:N; PRÉSTAMO total	Asociación	Bibliotecario 1 — 0..* Prestamo
GENERA	1:N; MULTA total	Composición	Prestamo 1 ◆— 0..* Multa
Atributo compuesto nombre	—	Composición	Usuario 1 ◆— 1 Nombre

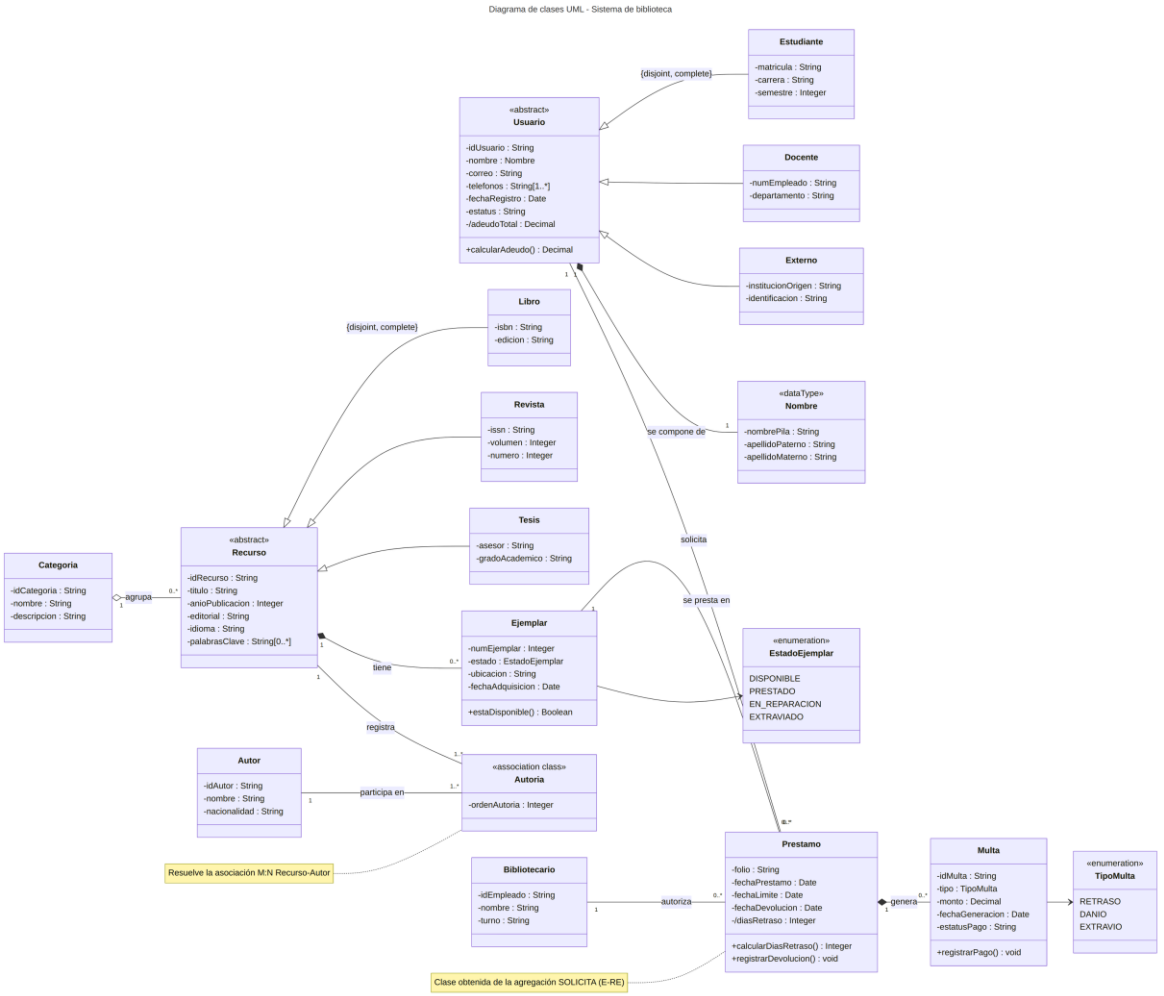
Nota. Elaboración propia. ◆ = composición; ◇ = agregación; ◁ = generalización.

5. Diagrama de clases UML

La Figura 2 presenta el diagrama de clases completo. Además de las clases derivadas de las entidades, incorpora operaciones, visibilidad de las propiedades y dos enumeraciones.

Figura 2

Diagrama de clases UML del sistema de biblioteca



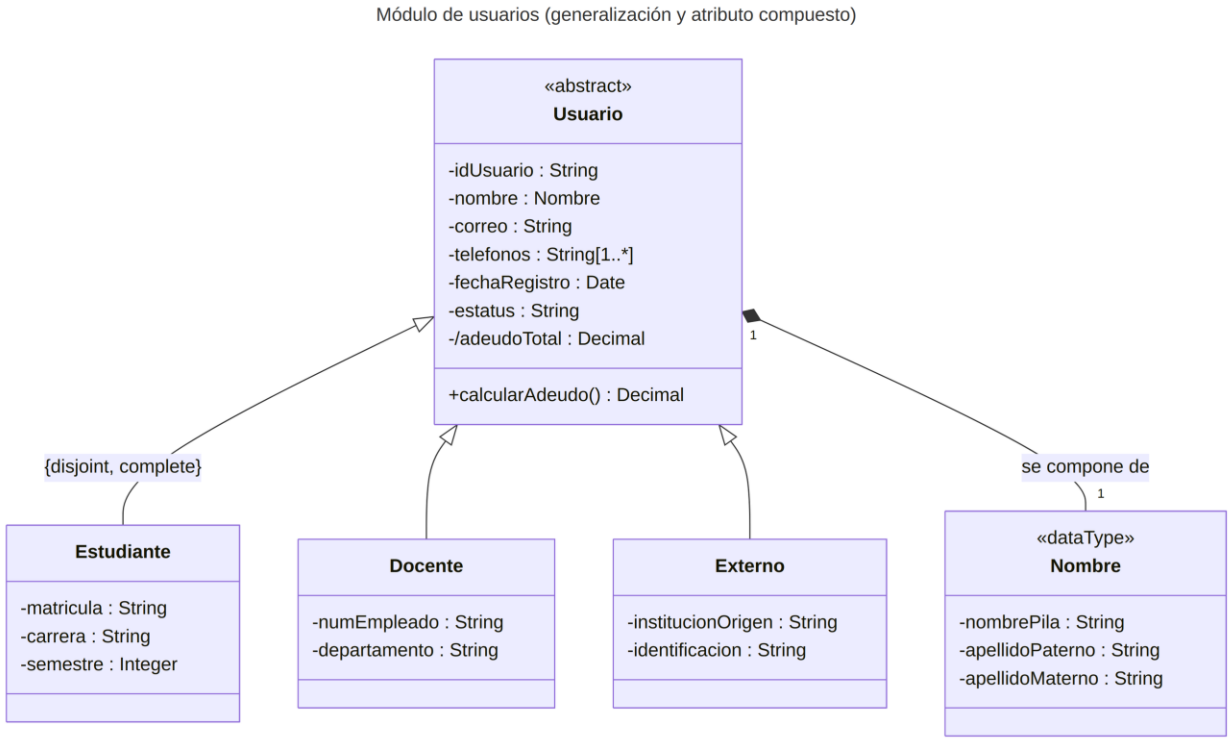
Nota. Elaboración propia con Mermaid (código fuente: diagrama_clases_uml_biblioteca.mmd).

5.1 Módulo de usuarios

La entidad USUARIO y sus tres subtipos se transformaron en una jerarquía de herencia con Usuario como clase abstracta, de modo que solo se instancian estudiantes, docentes o externos (generalización total y disjunta). El atributo compuesto nombre se convirtió en la clase de tipo de dato Nombre, unida a Usuario por composición.

Figura 3

Módulo de usuarios: herencia y composición



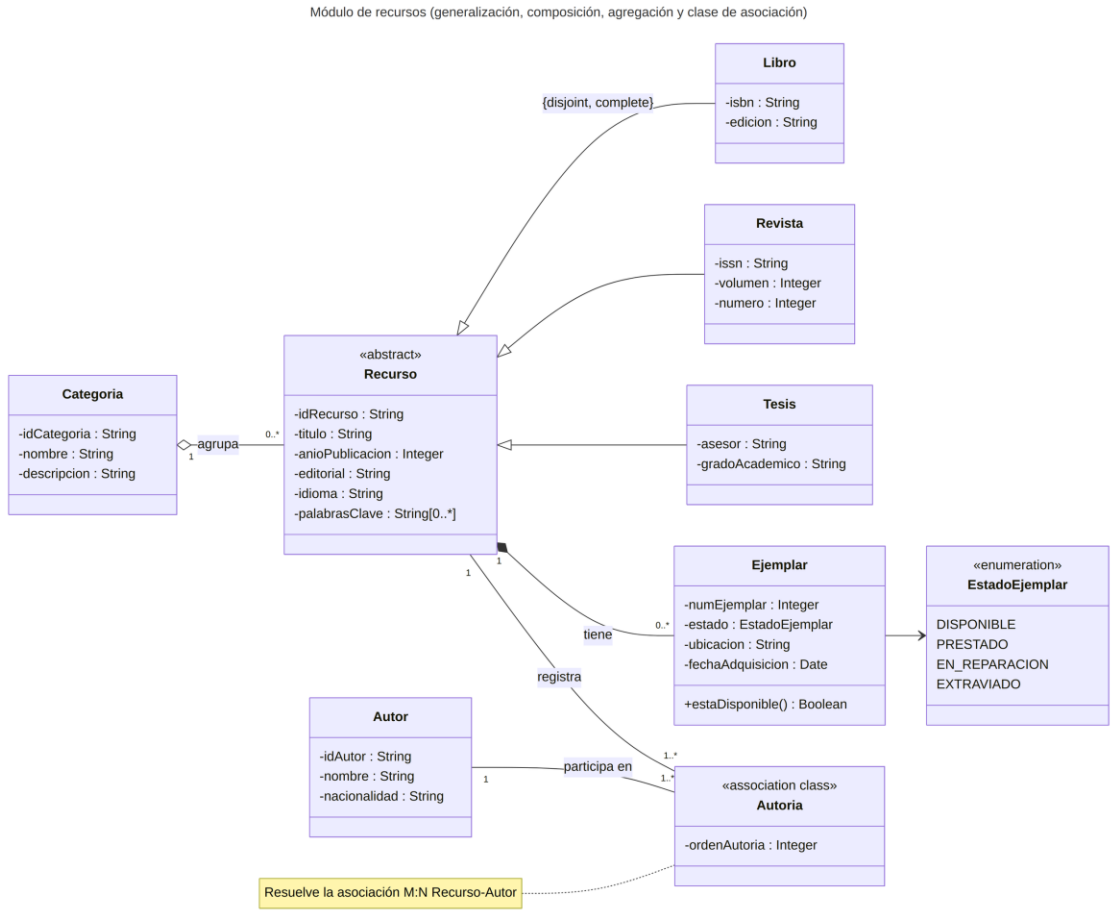
Nota. Elaboración propia con Mermaid (modulos/modulo_usuarios.mmd).

5.2 Módulo de recursos

Recurso es una clase abstracta con tres subclases. La entidad débil EJEMPLAR pasó a ser una clase unida a Recurso por composición, porque un ejemplar no existe sin su recurso. Categoria se relaciona con Recurso mediante agregación, pues una categoría agrupa recursos que siguen existiendo por sí mismos. La relación M:N ESCRIBE, que tiene el atributo orden_autoria, se modeló con la clase de asociación Autoria.

Figura 4

Módulo de recursos: herencia, composición, agregación y clase de asociación



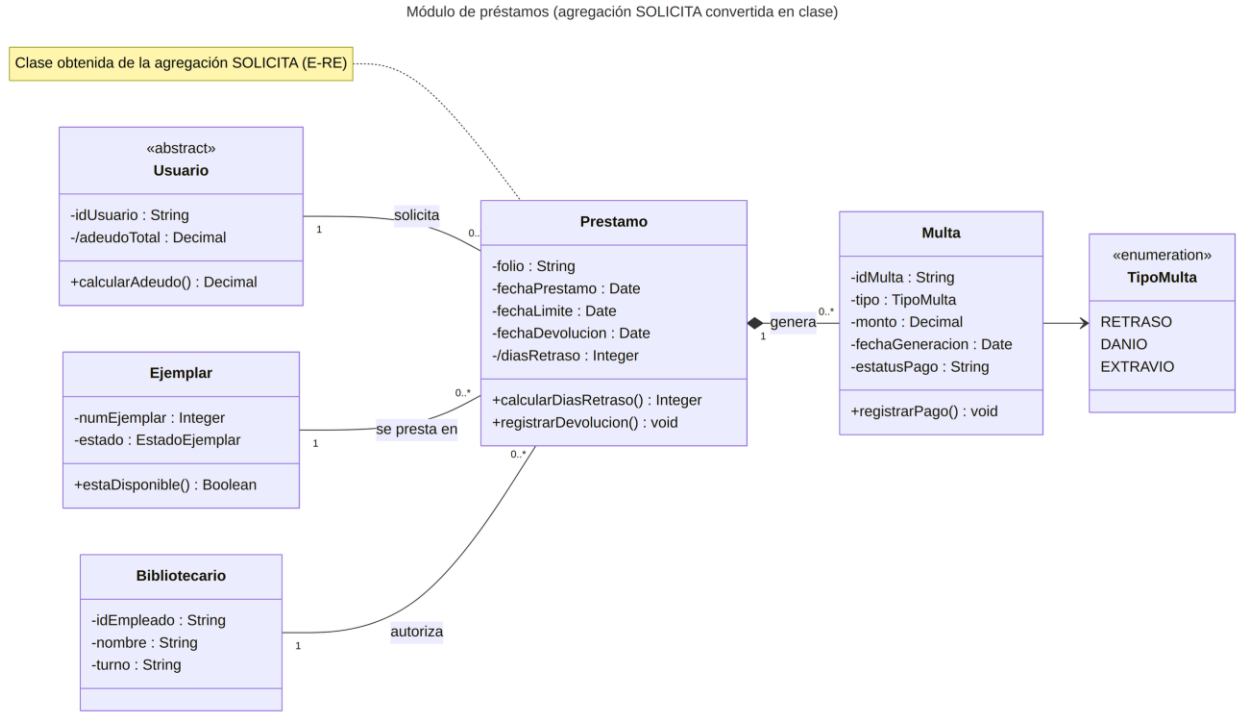
Nota. Elaboración propia con Mermaid (modulos/modulo_recursos.mmd).

5.3 Módulo de préstamos

La agregación SOLICITA del E-RE se convirtió en la clase Prestamo, que contiene el folio y las fechas, y se asocia con Usuario, Ejemplar y Bibliotecario. La relación GENERA se modeló como composición entre Prestamo y Multa, porque una multa solo existe ligada al préstamo que la originó. Los atributos derivados se representan con "/" y se acompañan de una operación de cálculo.

Figura 5

Módulo de préstamos: agregación convertida en clase y composición



Nota. Elaboración propia con Mermaid (modulos/modulo_prestamos.mmd). Se muestran solo algunos atributos de Usuario y Ejemplar.

6. Comparativa detallada: E-R vs. UML

La Tabla 5 compara ambos modelos según sus principales características.

Tabla 5

Comparativa entre el modelo E-R y el diagrama de clases de UML

Aspecto	Modelo E-R	UML (diagrama de clases)
Propósito	Representar los datos que se almacenarán y sus relaciones (diseño de bases de datos).	Representar la estructura y el comportamiento de un sistema de software orientado a objetos.
Nivel de abstracción	Conceptual, previo al diseño lógico y físico de la base de datos.	Puede usarse desde el análisis hasta el diseño detallado de las clases.
Elemento principal	Entidad	Clase
Atributos	Nombre y tipo (simple, compuesto, multivalorado, derivado); el tipo de dato suele omitirse.	Propiedades con nombre, tipo, visibilidad, multiplicidad y valor inicial.
Identificador	Clave primaria (atributo subrayado) y clave parcial.	No hay clave: cada objeto tiene identidad propia; el identificador se modela como una propiedad más.
Comportamiento	No se representa.	Operaciones con parámetros y tipo de retorno.
Visibilidad	No existe.	Pública (+), privada (-), protegida (#) y de paquete (~).
Relaciones	Relaciones con nombre entre entidades (rombos).	Asociaciones, dependencias, generalizaciones, agregaciones y composiciones.
Cardinalidad y participación	Cardinalidad (1:1, 1:N, M:N) y participación total o parcial (línea doble o simple).	Multiplicidad mínimo..máximo en cada extremo (por ejemplo 0..*, 1..*).
Herencia	Especialización/generalización del E-RE, con restricciones disjunta/solapada y total/parcial.	Generalización con restricciones {disjoint}, {overlapping}, {complete} e {incomplete}.
Todo-parte	Entidad débil y relación identificadora; la agregación del E-RE es otro concepto.	Composición (ciclo de vida dependiente) y agregación (todo-parte débil).
Atributos de relación	Se dibujan en la relación.	Se modelan con una clase de asociación.
Atributo derivado	Elipse punteada.	Propiedad precedida por una barra (/).
Navegabilidad	No existe; las relaciones son bidireccionales.	Las asociaciones pueden indicar la dirección de navegación.
Notación	Rectángulos, rombos y elipses (notación de Chen).	Cajas de clase con tres compartimentos (nombre, atributos y operaciones) y líneas con terminadores.
Uso posterior	Se transforma en el esquema relacional (tablas).	Se transforma en código de un lenguaje orientado a objetos.

Nota. Elaboración propia a partir de Chen (1976), Elmasri y Navathe (2016), Object Management Group (2017) y Booch et al. (2005).

7. Análisis de coincidencias y diferencias

7.1 Coincidencias

Ambos modelos describen la estructura de la información mediante elementos equivalentes, lo que hace posible una transformación casi directa (Tabla 6).

Tabla 6
Conceptos equivalentes entre ambos modelos

Concepto	En el E-R	En UML
Entidad / clase	Entidad con atributos	Clase con propiedades
Relación / asociación	Relación entre entidades	Asociación entre clases
Cardinalidad / multiplicidad	1:1, 1:N, M:N	1, 0..*, 1..*
Especialización / herencia	Supertipo y subtipos	Superclase y subclases
Atributo derivado	Elipse punteada	/atributo
Atributo multivalorado	Elipse doble	Propiedad con multiplicidad

Nota. Elaboración propia.

Además, los dos permiten expresar jerarquías de especialización, restricciones de cardinalidad y relaciones con dependencia de existencia, y ambos son independientes de una tecnología concreta en su nivel conceptual.

7.2 Diferencias

- **Comportamiento:** UML incorpora operaciones; el E-R solo describe datos. En el caso estudiado se agregaron calcularAdeudo(), calcularDiasRetraso(), registrarDevolucion(), registrarPago() y estaDisponible().
- **Identificadores:** el E-R basa la identificación en claves; en UML la identidad pertenece al objeto. Por ello la clave se conserva como una propiedad más, útil al guardar los objetos en una base de datos.
- **Tipos y visibilidad:** UML obliga a declarar el tipo de cada propiedad y su visibilidad, información que el E-R no maneja.
- **Relaciones M:N:** el E-R las representa directamente; en UML se resuelven con una clase de asociación o intermedia, como Autoria y Prestamo.
- **Participación:** en UML se expresa solo con la multiplicidad mínima, de modo que se pierde la distinción visual entre línea simple y doble.
- **Todo-parte:** UML distingue composición y agregación, mientras que el E-R solo identifica la dependencia de existencia mediante la entidad débil.
- **Navegabilidad:** UML puede indicar en qué sentido se recorre una asociación; en el E-R las relaciones no tienen dirección.

8. Implicaciones para el desarrollo orientado a objetos

El diagrama de clases sirve de puente entre el diseño de la base de datos y el código. Cada elemento se traduce a una estructura del lenguaje y a una estructura de la base de datos (Tabla 7). La diferencia entre el modelo de objetos y el relacional, conocida como desajuste de impedancia, obliga a decidir cómo se almacenan las jerarquías y las colecciones, y suele resolverse con herramientas de mapeo objeto-relacional (Elmasri y Navathe, 2016; Teorey et al., 2011).

Tabla 7

Correspondencia entre UML, código orientado a objetos y base de datos

Elemento UML	Implementación en un lenguaje orientado a objetos	Implementación en la base de datos
Clase	Clase del lenguaje (class)	Tabla
Propiedad	Atributo o campo de la clase	Columna
Generalización	Herencia (extends)	Una tabla por jerarquía, por clase o por subclase (estrategias de mapeo)
Composición	Objeto o colección contenida; se crea y destruye con el todo	Llave foránea con borrado en cascada
Agregación	Referencia o colección a objetos independientes	Llave foránea sin borrado en cascada
Asociación 1:N	Referencia en un lado y colección en el otro	Llave foránea en el lado N
Clase de asociación	Clase intermedia con referencias a las dos clases	Tabla intermedia con dos llaves foráneas y atributos propios
Propiedad multivalorada	Colección (List, Set)	Tabla aparte relacionada
Enumeración	Tipo enumerado (enum)	Columna con restricción de valores o tabla catálogo

Nota. Elaboración propia a partir de Teorey et al. (2011) y Elmasri y Navathe (2016).

A modo de ejemplo, parte del diagrama de clases se traduciría a código de la siguiente manera (en Java):

```
public abstract class Usuario {
    private String idUsuario;
    private Nombre nombre;                  // composición
    private List<String> telefonos;         // multiplicidad 1..*
    private List<Prestamo> prestamos;       // asociación 0..*
    public BigDecimal calcularAdeudo() { /* suma de multas pendientes */ }
}
public class Estudiante extends Usuario {   // herencia
    private String matricula;
}
public class Recurso {
    private List<Ejemplar> ejemplares;      // composición
}
```

Entre las ventajas de partir de este diagrama están que las reglas de negocio ya están capturadas como multiplicidades y herencia, que las operaciones quedan definidas junto a los datos que manejan y que se reduce el riesgo de inconsistencias entre el modelo de datos y el código.

9. Conclusiones

La transformación del modelo E-R extendido a un diagrama de clases UML es directa en la mayoría de los casos: las entidades pasan a ser clases, los atributos a propiedades, las especializaciones a herencia y las entidades débiles a composición. Las diferencias aparecen en los elementos que UML añade (operaciones, visibilidad, tipos, navegabilidad y enumeraciones) y en las relaciones M:N, que requieren una clase intermedia.

Ambos modelos se complementan: el E-R es la base para diseñar y normalizar la base de datos, y el diagrama de clases es la base para construir el software que la utiliza. Contar con los dos mantiene la coherencia entre los datos y la lógica de la aplicación.

Referencias

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language user guide (2.ª ed.). Addison-Wesley.

Chen, P. P.-S. (1976). The entity-relationship model—Toward a unified view of data. ACM Transactions on Database Systems, 1(1), 9–36. https://doi.org/10.1145/320434.320440

Connolly, T., & Begg, C. (2015). Database systems: A practical approach to design, implementation, and management (6.ª ed.). Pearson.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of database systems (7.ª ed.). Pearson.

Fowler, M. (2004). UML distilled: A brief guide to the standard object modeling language (3.ª ed.). Addison-Wesley.

Mermaid. (s. f.). Mermaid: Diagramming and charting tool [Software]. https://mermaid.js.org/

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), version 2.5.1 (formal/17-12-05). https://www.omg.org/spec/UML/2.5.1/

Teorey, T., Lightstone, S., Nadeau, T., & Jagadish, H. V. (2011). Database modeling and design: Logical design (5.ª ed.). Morgan Kaufmann.